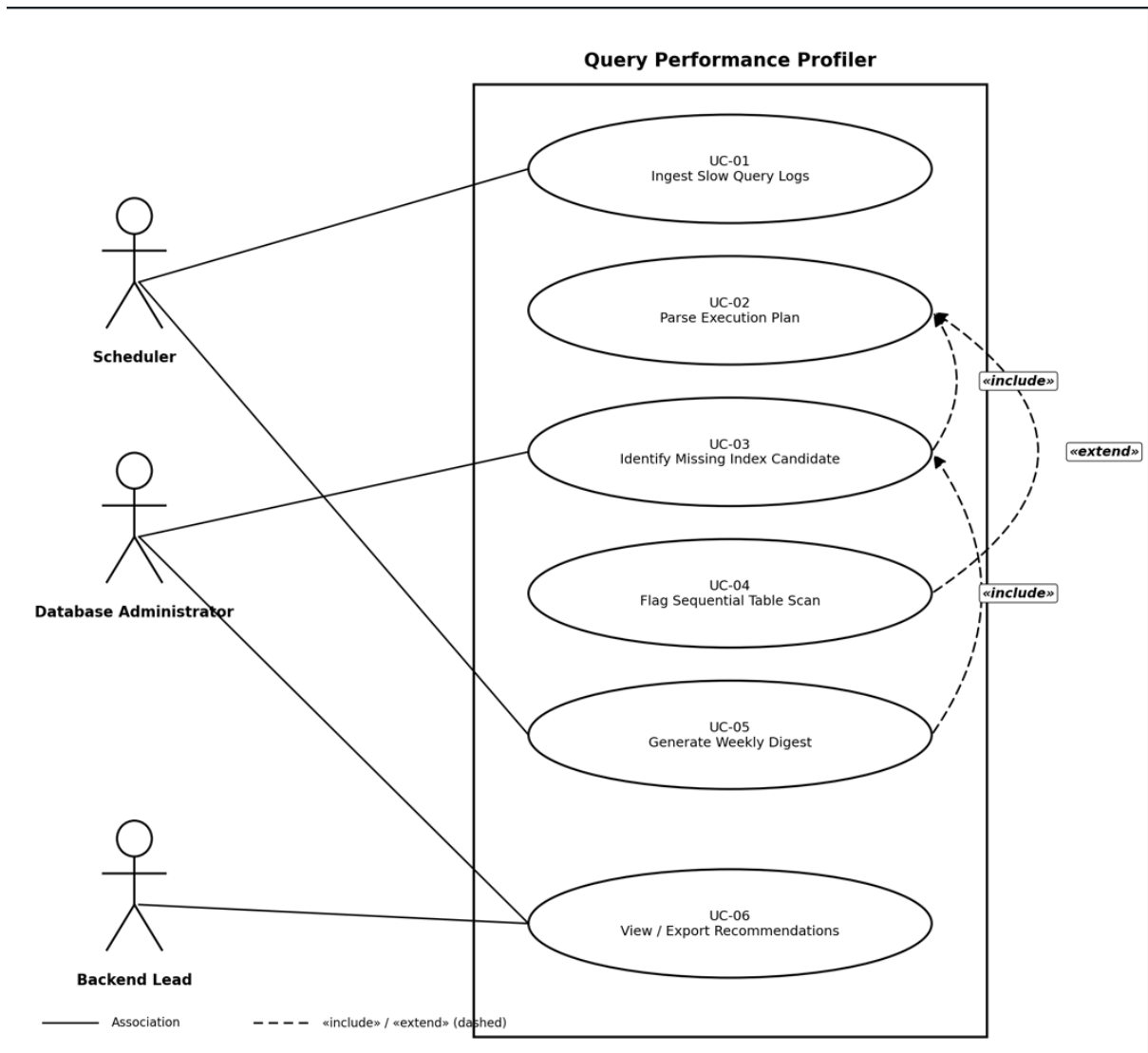




Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall parse Postgres/MySQL EXPLAIN plans, highlight sequential table scans on large tables, and recommend optimal index column definitions.	High	Pass: Missing composite index identified on filtered columns. Fail: Query plan with sequential scan marked optimized.	Core value proposition — turns raw execution plans into actionable index recommendations for the DBA.
FR-002	Functional	The system shall ingest slow query logs from configured Postgres/MySQL sources on a continuous or scheduled basis.	High	Pass: A log entry written to a monitored source appears in the system's query log store within 2 minutes. Fail: Entry is missing or delayed beyond 2 minutes.	Without reliable ingestion, no downstream analysis (parsing, index recommendation, digest) is possible.
FR-003	Functional	The system shall identify missing composite index candidates by analyzing filter, join, and sort columns across multiple related queries.	High	Pass: For a set of queries filtering/joining on the same 2+ columns without a covering index, exactly one composite index recommendation covering those columns is produced. Fail: No recommendation is produced, or a single-column index is recommended when a composite index would better cover the workload.	Composite index recommendations based on cross-query patterns give higher-value guidance than single-query analysis alone.
FR-004	Functional	The system shall generate a weekly performance optimization digest summarizing the top slow queries, flagged sequential scans, and index recommendations.	Medium	Pass: A digest is generated every 7 days containing at least the top 10 slowest queries and all open index recommendations from that period. Fail: Digest is not generated on schedule or omits required sections.	Gives the Backend Lead and DBA a periodic, digestible summary instead of requiring manual log review.
FR-005	Functional	The system shall allow the Database Administrator and Backend Lead to view, filter, and export flagged queries and index recommendations.	Medium	Pass: A user can filter recommendations by table, date range, or status, and export the filtered results as CSV/PDF. Fail: Filtering returns unrelated results or export fails/produces an empty file.	Enables stakeholders to act on recommendations outside the tool (e.g., in a migration script or ticket).
NFR-001	Nonfunctional (Performance & Security)	Slow query log ingestion must handle up to 5,000 log records per minute without dropping entries or slowing the host database.	High	Pass: Benchmarking tests confirm target latency and security standards under simulated peak load. Fail: Any entry is dropped, or host DB query latency increases beyond baseline during ingestion.	Protects production database performance while still capturing a complete, trustworthy log dataset.
NFR-002	Nonfunctional (Security & Availability)	Access to ingested query logs, execution plans, and index recommendations must be role-restricted (DBA/Backend Lead only) and encrypted at rest and in transit.	High	Pass: Penetration/config test confirms unauthorized roles cannot read logs or recommendations, and data-at-rest/in-transit encryption is verified. Fail: Any unauthorized role can access the data, or data is stored/transmitted unencrypted.	Query logs and execution plans can reveal sensitive schema and data-access patterns, so access must be restricted to authorized stakeholders.

Use Case: Identify Missing Index Candidate

Primary Actor: Database Administrator

Secondary Actor: Scheduler (triggers the analysis automatically as part of the weekly digest)

Preconditions: The query's slow query log entry has already been ingested and its EXPLAIN plan has already been parsed, so structured plan data (scanned tables, filter/join/sort columns, row estimates) is available.

Postconditions: A recommended composite index is generated and stored and linked to the query, or the query is marked "optimized, no action needed."

Main Success Scenario

1. Database Administrator opens the query analysis screen and selects a logged query to review.
2. System retrieves the parsed EXPLAIN plan for the selected query.
3. System checks the plan for a sequential scan on a table above the large-table size threshold.
4. System confirms a sequential scan was found on a large table.
5. System extracts the filter, join, and sort columns the query uses on that table.
6. System compares those columns against similar queries run recently on the same table.
7. System checks the table's existing indexes against the extracted columns to see what is already covered.
8. System determines the existing indexes do not fully cover the columns.
9. System builds a recommended composite index definition (table, columns, and column order).
10. System saves the recommendation and links it to the query log entry.
11. System displays the recommendation and its estimated performance benefit to the Database Administrator.
12. Use case ends successfully.

What Happens If It Fails

If the system does not find a sequential scan on a large table (step 3), or if an existing index already fully covers the query's columns (step 8), the following happens instead:

1. System stops the index recommendation analysis for this query.
2. System marks the query as "optimized, no action needed" or "no large-table scan found," as appropriate.
3. System logs this result against the query so it still appears in the weekly digest.
4. System displays the result to the Database Administrator instead of a recommendation.
5. Use case ends without producing an index recommendation.